



Verilog HDL Project Report

Priyansh Lathigara – 202201449

Meet Zalavadiya – 202201482



WASHING MACHINE

❖ **Overview :**

- FPGA-based implementation of a washing machine using FSM.
- Automates stages like checking the door, filling water, adding detergent, washing, draining, and spinning.
- FSM logic written in Verilog and verified through simulation.
- Deployed on FPGA for real-time testing.
- Demonstrates hardware-optimized control for appliance automation.

❖ **Introduction :-**

➤ **Objective :**

- Automate washing machine operations using FSM on FPGA.
- Control operations like door check, water fill, washing, and spinning systematically.

➤ **Scope :**

- Translate FSM logic into Verilog for simulation and FPGA synthesis.
- Focus on modular and efficient operation design.
- Ensure reliable performance for real-time testing.

➤ **Background :**

- FSMs are widely used for sequential control in digital systems.
- FSM consists of states, transitions, and conditions for moving between states.
- This project applies FSM principles to implement washing machine automation.
- Verilog HDL is used for FSM implementation on FPGA.

❖ FSM Design :-

➤ Overview :

- FSM controls washing machine stages in a predefined sequence.
- States represent specific phases of operation (e.g., fill water, spin).
- Transitions occur based on input signals like sensors and timers.

➤ States and Transitions :

▪ Check Door

- Ensure door is closed before operation starts.
- Transition to *Fill Water* when door is closed.

▪ Fill Water

- Fill the drum with water.
- Transition to:
 - ♦ *Add Detergent* when water is filled.
 - ♦ Self-loop if water is not yet filled.

▪ Add Detergent

- Add detergent to the drum.
- Transition to *Cycle* when detergent is added.

▪ Cycle

- Perform the washing operation.
- Transition to *Drain Water* after the cycle is complete.

▪ Drain Water

- Drain water after washing.
- Transition to *Spin* when water is fully drained.

▪ Spin

- Spin clothes to remove excess water.
- Transition back to *Check Door* to restart or end the process.

➤ State Transition Logic :

- **Inputs:** Reset, doorClose, start buttons, water level detector, add detergent, cycleTimeout, waterIsdrained, spinTimeout.
- **Outputs:** Control signals for actuators (doorLock, motorStart, draining water etc).

- **Triggers:**

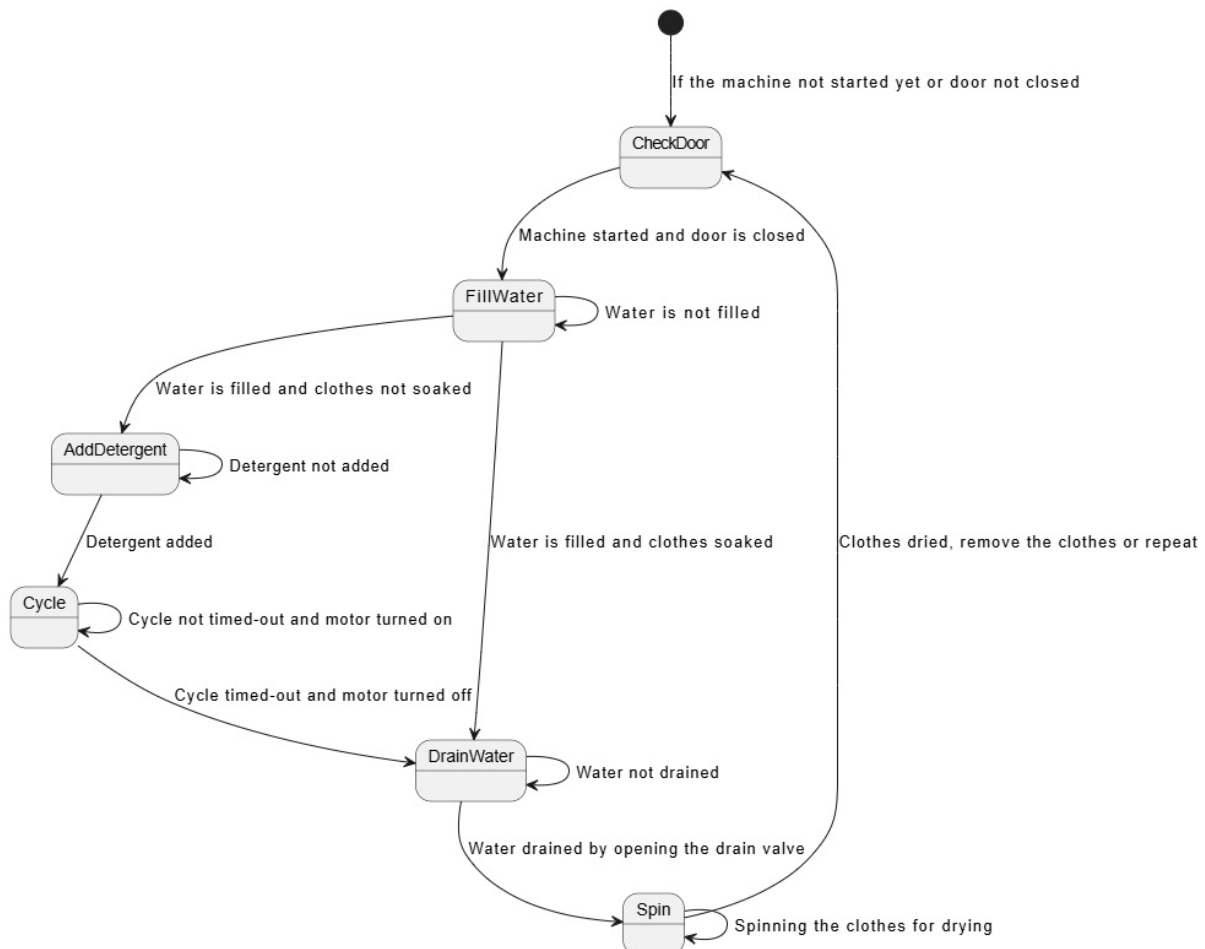
- Door closed → Move to *Fill Water*.
- Water level full → Move to *Add Detergent*.
- Detergent added → Move to *Cycle*.
- Cycle complete → Move to *Drain Water*.
- Water drained → Move to *Spin*.
- Spinning complete → Move to *Check Door*.

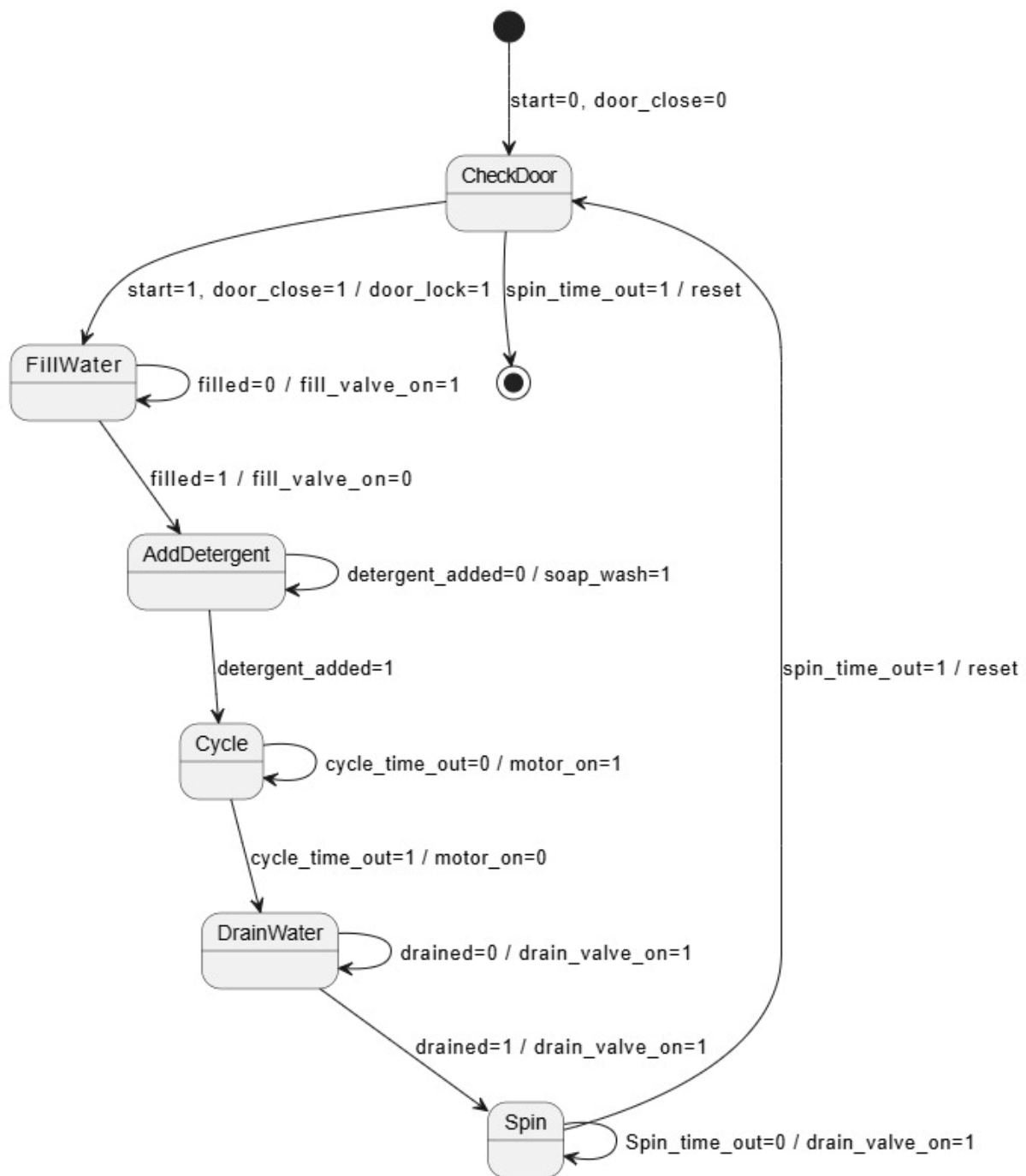
- **Challenges :**

- Handling edge cases like door left open or insufficient water level.
- Ensuring robust operation through self-loops and fallback states.

- **FSM Diagram :**

- Visual representation of states and transitions.
- Shows logical flow of operations in the washing machine





❖ Verilog Implementation :

➤ CODE :

⇒ Main Module :

```
module washing_machine(  
    input clk,  
    input reset,  
    input door_close,  
    input start,  
    input filled,  
    input detergent_added,  
    input cycle_timeout,  
    input drained,  
    input spin_timeout,  
    output reg door_lock,  
    output reg motor_on,  
    output reg fill_value_on,  
    output reg drain_value_on,  
    output reg done,  
    output reg soap_wash,  
    output reg spinning  
);  
  
    // Defining states  
    parameter check_door    = 3'b000;  
    parameter fill_water    = 3'b001;  
    parameter add_detergent = 3'b010;  
    parameter cycle         = 3'b011;  
    parameter drain_water   = 3'b100;  
    parameter spin          = 3'b101;  
  
    reg [2:0] current_state, next_state;  
  
    // Combinational Logic for state transitions and outputs  
    always @(*) begin  
        // Default values for outputs  
        next_state = current_state;  
        motor_on = 0;  
        fill_value_on = 0;  
        drain_value_on = 0;  
        door_lock = 0;  
        soap_wash = 0;  
        spinning = 0;  
        done = 0;  
  
        case (current_state)
```

```

check_door: begin
    if (start && door_close) begin
        next_state = fill_water;
        door_lock = 1;
    end
end

fill_water: begin
    door_lock = 1;
    fill_value_on = !filled;
    if (filled) begin
        if (!soap_wash) begin
            next_state = add_detergent;
            soap_wash = 1;
        end else begin
            next_state = cycle;
            soap_wash = 1;
        end
    end
end

add_detergent: begin
    door_lock = 1;
    soap_wash = 1;
    if (detergent_added) begin
        next_state = cycle;
    end
end

cycle: begin
    door_lock = 1;
    motor_on = 1;
    if (cycle_timeout) begin
        next_state = drain_water;
    end
end

drain_water: begin
    door_lock = 1;
    drain_value_on = !drained;
    if (drained) begin
        next_state = spin;
        soap_wash = 1;
        spinning = 1;
    end
end

spin: begin

```

```

        door_lock = 1;
        motor_on = 1;
        if (spin_timeout) begin
            next_state = check_door;
        end
    end

    default: begin
        next_state = check_door;
    end
endcase
if(motor_on & fill_value_on & drain_value_on & door_lock & soap_wash &
spining) begin
    done = 1;
end
end

// Sequential logic for state update
always @(posedge clk or posedge reset) begin
    if (reset) begin
        current_state <= check_door;
    end else begin
        current_state <= next_state;
    end
end
endmodule

```

[Project-VideoLink](#)

❖ Conclusion :

- The implementation of the washing machine finite state machine (FSM) on FPGA was successfully achieved.
- The FSM design covered all essential states, such as **Check Door, Fill Water, Add Detergent, Cycle, Drain Water, and Spin**, ensuring functionality similar to a real washing machine.
- Verilog was utilized effectively to model and simulate the FSM, allowing precise control of each state transition.
- This project demonstrated how FPGA-based implementations can efficiently control real-world systems by leveraging its **flexibility**.